Risk Assessment of Insurence Policies using Machine Learning Techniques

Lukáš Trstenský, Bartosz Kochanski and Maia Ghionea

https://github.com/SAKULAK/ITU-MachineLearning-FinalProject-Couriers

Contents

1 Introduction

This paper aims to explore methods to model the claims risk of insurance policies using various machine learning methods as well as discuss their implementation. We will explore and clean the data, then define the claims risk based on the data available in the dataset.

1.1 Machine Learning Methods

In this paper we will use custom implementation of a Feed-Forward Neural Network Regressor and custom implementation of a Decision Tree Regressor using only Python numerical libraries, e.g. NumPy. Furthermore, we will use an ensemble method using a combination of **TBD**

2 Data

The dataset used in the scopes of this project is the French Motor Third-Part Liability (`freMTPL2freq`) dataset from the `CASdatasets` (Dutang et al., 2024, p. 71) R package. It contains numerical and categorical information on the insurance claims of drivers for their cars spanning multiple regions, details about the cars themselves, such as age, gas type or brand, the drivers, and even the regions where the cars are driven, which more or less impact the risk of a claim being made.

2.1 Data Format

For each unique policy in our dataset, there is a number of claims made within the policy, its duration, then subsequent information on the driver and car. For a clearer overview of the set see Table 1 for the names and corresponding descriptions of the data columns.

Table 1: Data columns and descriptions.

Name	Description
IDpol	Unique policy ID.
ClaimNb	Number of claims made in policy.
Exposure	Duration of policy in years.
Area	Density category of driver's residence from "A" for rural area to "F" for urban centre.
VehPower	Power category of car.
VehAge	Age of car.
DrivAge	Age of driver.
BonusMalus	Bonus/malus, between 50 and 350: < 100 means bonus, > 100 means malus.
VehBrand	The car brand divided in the following groups: A- Renaut Nissan and Citroen, B- Volkswagen, Audi, Skoda and Seat, C- Opel, General Motors and Ford, D- Fiat, E- Mercedes Chryslerand BMW, F- Japanese (except Nissan) and Korean, G- other.
VehGas	Type of gas that the vehicle uses.
Density	Number of inhabitants per km^2 in the city the driver of the car lives in.
Region	The policy region in France (based on the 1970-2015 classification).

2.2 Data Cleaning

A conservative approach was taken in the scope of data cleaning, by minimally modify the data. The material did not present any obvious signs of subverting our assumptions, except for the following:

1. Exposure values should lie in the range between 0 and 1; since the data was collected over the span of one year, we assume that any value above 1 is a mistake in the collection and therefore any value over this range will be reduced to 1, the maximum value permitted.

2. Any data points where $VehAge > 30$ are excluded. This is the cut-off mark at which vehicles become classified as historic cars and will in turn be treated differently by car insurance companies.

Despite the rest of the data not presenting any faults, the information left is narrowed down to solely the figures utilized by the models. Namely, the following columns have been dropped: (**EDIT THESE IF CHANGES**)

a. `IDpol` - as it is not a feature

b. `Area` - categorical counterpart of `Density`, therefore not needed

c. `VehBrand` - Vehicle brand alone is not an indicator of policy risk, especially since a singular brand has many models of vehicle, potentially attracting different driver types, however, we assume that the risk evens out along the whole brand's lineup of cars.

d. `Region` - another indicator of `Density`.

2.3 Visualization

As seen in Figure 3 the dataset is heavily biased towards 0 claims. This can pose a significant issue by cluttering the feature space with noise, especially if the differences in feature values for data points with higher numbers of claims and the data points with 0 number of claims are small.

Figure 1: Histogram of the number of claims.

Figure 2: This is an interesting visualization of the data.

3 Features

The data columns left after the data cleaning are: `VehPower`, `VehAge`, `DrivAge`, `BonusMalus`, `VehGas`, `Density`, and these will be used as the features for our model, the variables used to teach the algorithms the necessary patterns for the prediction.
The columns on which we base our `Risk` calculations are the following: `ClaimNb` and `Exposure`, therefore not included in the feature set. These will steer the predictions in the correct way, this will be the information the models are to predict.

3.1 Claims Risk

How do we calculate it, why (idk)
Very imbalanced Figure 3

Figure 3: Histogram of Risk values

3.2 PCA & Clustering

Clustering is defined as an unsupervised learning method, which groups the observations made based on similarity, with the goal of identifying homogeneous subgroups of insurance policy claimers; telling also by the name.

For better clustering, the dimensions of the feature space should be reduced, which is why the method called PCA was introduced in this project. P(rincipal) C(omponent) A(nalysis) is a technique of dimensionality reduction which aims to turn the correlated numerical variables into a smaller set of uncorrelated ones. Where, before, the numerical features of `DrivAge`, `BonusMalus` or `Density` could be correlated, after applying the PCA, the data's high dimensionality will be stabilized and any multi-collinearity will be reduced, as the resulting set should be smaller, uncorrelated. Further data cleaning ensued for the specific requirements of PCA, such as excluding the categorical variables and standardizing the data to prevent the dominion of large scaled values over the analysis. Four principal components need to be used to retain 89% of the data; which means that without significant loss most of the original information is represented in a lower dimensional space. As such, global structure is preserved and variance is recognized in fewer dimensions.

Figure 4: PCA with Risk as color — Clustering on PCA data.

The clustering is the following step and from various methods, we applied K-Means clustering on our data, as it's finely compatible with continuous, scaled data, resulting from the principal component analysis. The main idea behind this clustering method is the partitioning of the data into `k` clusters, `k` being the number of such groups created, through minimizing the variance between different clusters. To find the right value for `k`, a silhouette score was used as an evaluation metric when trying different values. This score primarily measures similarity of each observation to their own cluster and the quality of the separation between clusters, after which the best one is chosen. The `k` which maximized the number of the silhouette score, is the one that is chosen for the final `k` split. In summary of the vein of clustering, meaningful behavior and risk patterns will be displayed together, letting the visualization speak for different groups of the insurance customers.

4 M1 - Decision Tree

A decision tree is essentially a flowchart for making predictions. It works by sorting the data into smaller and smaller groups. In decision tree terminology those groups are called leaves. They are created to be as ”pure” as possible using a functions that can quantify said purity.
Once the tree is built every leaf node is assigned a value used to predict the target variable of new, unseen data.
For Regression: The leaf is assigned the Average (Mean) of the values in the leaf.
For Classification: The leaf is assigned the Majority Class. In other words the class that is most common in the leaf. This decision is made utilizing certain principles for tie-breaking.

4.1 Structure

The structure of this project's Decision Tree implementation follows a classic object oriented design. It is constructed with nodes witch function as either:
leaf - which holds the prediction
decision node - which holds the splitting rule (the feature and threshold) as well as pointers to it's left and right children.

The DecisionTree class manages the logic of how the tree is built and how it makes decisions. It stores the final structure made of nodes, which is similar to a binary tree, where each node points to either nothing or it's two children nodes.
The DecisionTree is structured in a way that allows it to function as both classifier and regressor based on the values of the observed variables.

4.2 Finding Splits

The most important part of constructing a decision tree is finding a split. This is the most computationally heavy part. The Decision Tree:

1. Loops through every feature
2. Sorts the data by feature
3. Tests every possible midpoint between values as a potential ”threshold.”
4. Calculates the Impurity Decrease for each using the selected criterion. It remembers the one that reduced the disorder the most.

Depending on the parameters the tree is built from this in one of two ways.

4.2.1 Recursive

The standard way of building a tree. The algorithm starts at the Root. It splits the Root into a Left and Right child. It then moves to the Left child and tries to split it. If it can, it moves into that child's Left child and repeats the process.
As consequences of that It dives as deep as possible into the leftmost branch until it hits a stopping condition. Only then does it backtrack to fill in the right-hand side branches.

4.2.2 Priority Queue

This is the method used when the tree is limited by the total number of leaves. The recursive approach would not work with such limitation. Instead of diving deep into one branch, the algorithm looks at all current leaf nodes across the entire tree. The priority queue (heap) stores potential ”Impurity Decrease” for every leaf.
The algorithm always removes from the queue the node that offers the highest impurity decrease, regardless of where it is in the tree or how deep it is.

4.3 Stopping Parameters

Why, what problems they solve

Maybe an enumerate of stopping params with simple description of how they work, if that fits, if not use subsubsection for each stopping parameter

a. **Max Depth**
   This is the description?

b. **Minimum Samples to Split** -

c. **Minimum Samples at Leaf** -

d. **Maximum number of Leaves** -

e. **Minimum Impurity Decrease** -

5 M2 - Feed-Forward Neural Network

A Feed-Forward Neural Network (*FFNN*) is our second custom implemented regression method. In this section we explain the basics of how a FFNN works along with how an implemention might look.

5.1 The Structure

A FFNN is a collection of nodes organized in layers. First, the input layer. The input layer number of nodes is equal to the number of features in the dataset. Second, the hidden layers. There can be an arbitrary number of hidden layers and of nodes per hidden layer. Generally, the more hidden layers or nodes per layer the more complex the Neural Network becomes, which makes it more prone to overfitting and more computationally heavy to train and run predictions on. Lastly, the output layer. The number of nodes in the output layer depends on the task, but for regression there is just one node, which give the value predicted value for the data points.

Figure 5: Structure of a FFNN

5.2 The Flow

A FFNN at its core follows a very simple flow, see Figure 6, for each iteration or epoch data is run through the network to gain initial predictions, a cost function is calculated, weights are updated based on their partial derivate of the cost function.

Figure 6: Flowchart of a FFNN

5.3 Initialization

We initialize an array of weights (W_{ij}) and an array of biases (b_j) for each pair of subsequent layers. The shape of W_ij is the number of nodes in the previous layer (i) by the number of nodes in the current layer (j) which is also the shape of the bias array.

To fill the arrays with initial values we use the He Initialization (He et al., 2015) by drawing from a zero-mean normal distribution with standard deviation of $\sqrt{2/n_i}$ which is considered to be among the best initializations for shallow ReLU based FFNNs.

5.4 Forward Pass

During the Forward Pass, the data, as the name suggests propagates forward through the network. At each layer, except the input layer, the activations of the previous layer (a_i) are weighed by the weights and have the bias added to them:

$$z_j = a_i W_{ij} + b_j$$

Then they are passed through the activation function to become the activation of the current layer, which are then passed to the next layer.

5.4.1 Activation Functions

There are many activation functions that one might choose depending on the task. Usually you use two different activation functions, one for the hidden layers and a different one for the output layer. For hidden layers we use the widely used activation function, the Rectified Linear Unit (*ReLU*) function defined as $ReLU(x) = max(0, x)$. For the output layer of regression FFNNs a simple linear function is used $f(x) = x$

5.5 Backpropagation

Backpropagation is the step of the training process, which allows the model to learn. By utilizing gradient descent in the cost function landscape the weights can be updated in the direction and magnitude which results in the updated weights producing a less costly prediction. However, to be able to use gradient descent we need the partial derivatives of the cost function relative to each weight. This can be done fully symbolically, which for large networks is infeasible, but because of the chain rule, this can be simplified into a recursive process in which the derivative of the cost function relative to the weights of the output layer (j) is computed:

$$\frac{\partial C}{\partial W_{ij}} = \frac{\partial z_j}{\partial W_{ij}}(\frac{\partial C}{\partial z_j}) = \frac{\partial z_j}{\partial W_{ij}}(\frac{\partial a_j}{\partial z_j}\frac{\partial C}{\partial a_j}) =$$

$$a_i(\Phi'(z_j)C'(a_j))$$

The error term of the partial derivate for output layer for $\Phi = f(z_j) = z_j$ and $C = MSE = (y_{pred} - y_{true})^2/n$ is:

$$\frac{\partial C}{\partial z_j} = (1\frac{2(a - y_{true})}{n})$$

Then the effects of the weighted sum of the previous layer, in this case the last hidden layer (i), on the cost function is calculated:

$$\frac{\partial C}{\partial W_{ui}} = \frac{\partial z_i}{\partial W_{ui}}[\frac{\partial C}{\partial z_i}] = \frac{\partial z_i}{\partial W_{ui}}[\frac{\partial a_i}{\partial z_i}\frac{\partial z_j}{\partial a_i}(\frac{\partial C}{\partial z_j})] =$$

$$a_u[\Phi'(z_i)W_{ij}(\frac{\partial C}{\partial z_j})]$$

where $\Phi = ReLU$ then $\Phi' = 1\{z \geq 0\}$.
This holds true for any layer going backwards from the output layer. Its partial derivative has same

terms, only depending on its own parameters and the error term of the next layer ($\frac{\partial C}{\partial z_j}$).
The same derivation is done for the bias term, however to maintain its correct dimensions the contributions of each data point for a singular bias term are averaged and $\frac{\partial C}{\partial b_i} = \frac{\partial z_i}{\partial b_i}[\text{error term}] = 1[\text{error term}]$.

5.6 Updating Parameters

Updating parameters means taking a step in the parameter space in the direction of the weight gradient. Naively this can be done as $W_{t+1} = W_t - \alpha\nabla C(W_t)$, similarly $b_{t+1} = b_t - \alpha\nabla C(b_t)$, t being epoch number α learning rate, however this presents multiple issues, primarily using a constant α can cause exploding gradients, inability to reach the minima, and slow learning. For faster and more precise learning the Adaptive Moment Estimation (Adam) (Kingma and Ba, 2017) method was implemented.
Adam expands on Momentum (Polyak, 1964; Rumelhart et al., 1986) and RMSprop (Tieleman and Hinton, 2012). It utilizes two exponentially weighed moving averages (m_t and v_t) of the weight gradients to dynamically control the magnitude of learning rates for each weight. The first momentum mainly controls the direction of movement:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1)\nabla C(W_i)$$

β_1 usually set to 0.9, which means the current magnitude is composed of 90% of the previous momentum and only 10% the current gradient.

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2)\nabla C(W_i)$$

β_2 is usually set to 0.999. Since m_0 and v_0 are initialized as 0, then during early training they are small for which we account by bias adjusting:

$$\hat{m_t} = \frac{\hat{m_{t-1}}}{1 - \beta_1^t}, \hat{v_t} = \frac{\hat{v_{t-1}}}{1 - \beta_2^t}$$

for small t small denominator boosts momenta signals, while during late training the denominator being close to one barely has any effect on the momenta leaving the work to the moving averages. Then finally update parameters:

$$W_{t+1} = W_t - \alpha\frac{\hat{m_t}}{\sqrt{\hat{v_t} + \epsilon}}\nabla C(W_i)$$

The term $\frac{\hat{m_t}}{\sqrt{\hat{v_t}+\epsilon}}$ is effectively a scaling factor for the learning rate, adjusting it to be smaller when the variability in the cost function landscape is high and making it larger when the wanted change in direction is large. But by using the moving averages the descent is smooth without unnecessarily straying from the right path.

5.7 Non-Essential Features

For better flexibility and performance of the model regularization and early stopping were implemented.

5.7.1 Early Stopping

The training data is randomly split into a training and a validation set with the size of the validation set being 10% of the initial data.
After each epoch the cost is calculated on the validation set, if the cost increases for n consecutive epochs training is aborted and weights are set to be the weights from the best performing epoch. This helps to optimize for performance while allowing for lower training time.

5.7.2 Weight Decay

Decoupled weight decay was implemented for the Adam optimizer by adding an L_2 norm regularization term to the parameter update step:

$$W_{t+1} = W_t - \alpha(\frac{\hat{m_t}}{\sqrt{\hat{v_t} + \epsilon}}\nabla C(W_i) + \lambda\|W_t\|^2)$$

By adding the regularization penalty, large weights are pushed to be smaller forcing a smoother decision boundary, which is less capable of overfitting to noise and more likely to capture the underlying function.

6 Correctness Testing

To validate our implementation of the Decision Tree (*DT*) and the Feed-Forward Neural Network (*FFNN*) correctness tests were performed. For both the DT and the FFNN the basic implementation was tested in addition to being tested with all extra features enabled separately and all at once.

6.1 Comparing Against a Dummy Regressor

Using an insurance dataset (Lantz, 2013; Choi, 2018) with categorical features `sex`, `smoker`, `region` and numeric variables `age`, `bmi`, `children` with target variable `charges` the DT and FFNN models were compared to a null model predicting the mean of the target variable. All models were evaluated using MSE and all our model configurations beat the null model.

6.2 Simple Function Test

A sine funciton: $f(x) = sin(x) + N(0, 0.5)$ and a linear function with bias: $f(x) = 1 + 2x + N(0, 0.5)$ were used to compare our DT and FFNN implementations to an established implementation from the Scikit Learn (Pedregosa et al., 2011) (*sklearn*) Python package. All hyperparameters were set to be the same arbitrary values. Our FFNN implementation performed closely to the sklearn implementation, while our DT performed exactly the same. Figure 7 Figure 8

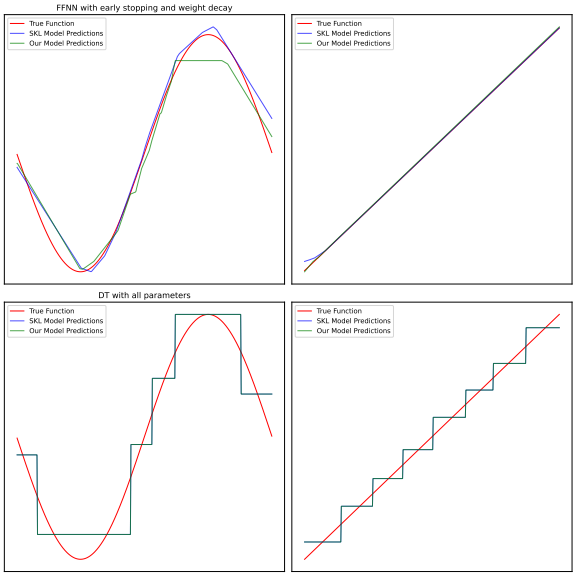

Figure 7: Ecample of comparison of our DT and FFNN implementations compared to the sklearn implementaion with continous feature.

7 M3 - Ensemble

Idk let's see what we do

8 Results

Probably some table of results of each method and how and why we evaluated like we did.

9 Limitations

Probably could have somehow cleaned the data better to get better separation of Risk values and would like claim amounts.

10 Conclusion

Hopefully something good. Summarize the results of each method, include running times say which one worked best as a combination of time and performance

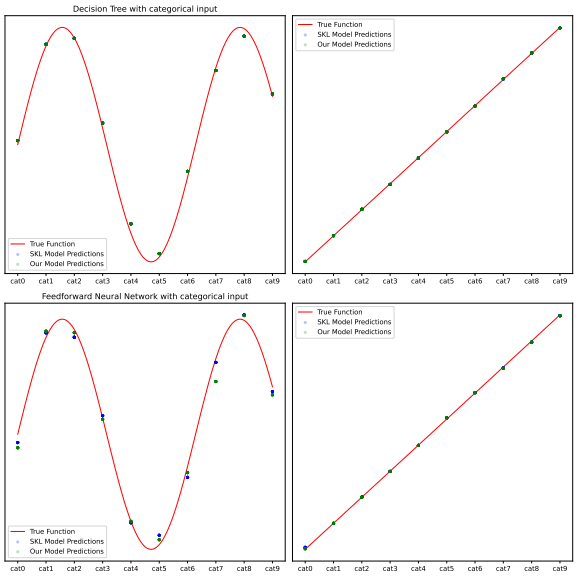

Figure 8: Comparison of our DT and FFNN implementations compared to the sklearn implementaion with categorical feature.

10.1 Improvments

everything

References

[Choi, 2018] Choi, M. (2018). Medical cost personal datasets. https://www.kaggle.com/datasets/mirichoi0218/insurance. Accessed: 2025-05-22.

[Dutang et al., 2024] Dutang, C., Charpentier, A., Gallic, E., and Siharath, J. (2024). CASdatasets: Insurance datasets. https://cas.uqam.ca/pub/web/CASdatasets-manual.pdf. R package version 1.2-0.

[He et al., 2015] He, K., Zhang, X., Ren, S., and Sun, J. (2015). Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. https://arxiv.org/abs/1502.01852.

[Kingma and Ba, 2017] Kingma, D. P. and Ba, J. (2017). Adam: A method for stochastic optimization. https://arxiv.org/abs/1412.6980.

[Lantz, 2013] Lantz, B. (2013). Machine learning with R. https://github.com/stedy/Machine-Learning-with-R-datasets.

[Pedregosa et al., 2011] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Édouard Duchesnay (2011). Scikit-learn: Machine learning in python. http://jmlr.org/papers/v12/pedregosa11a.html.

[Polyak, 1964] Polyak, B. T. (1964). Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17.

[Rumelhart et al., 1986] Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323(6088):533–536.

[Tieleman and Hinton, 2012] Tieleman, T. and Hinton, G. (2012). Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude.